

Trainer Teaching Guide: Java Collections, Class Relationships & File I/O

A detailed, textbook-style script for [java-collections-theory.html](#). Simple English, examples, analogies and natural fillers - read it aloud and it sounds like teaching, not reading.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [java-collections-theory.html](#) (24 slides)

HOW TO USE THIS GUIDE:

- ON SCREEN tells you what the slide shows.
- SAY is the full teaching script - read it word-for-word; fillers and questions are built in.
Pause where you see '...'.
- EMPHASISE / DELIVERY TIP are notes for you - do NOT read these aloud.

Slide 1 - Title: Collections, Class Relationships & File I/O

ON SCREEN

The blue title slide with the session name and the line 'three ways to manage data'.

SAY (read this aloud)

Good morning everyone, and welcome. Today's session has a slightly long name - Collections, Class Relationships, and File Input-Output - so let me tell you the one simple idea that ties all three together. Everything we do today is about one thing: managing data.

Think about it. A program is really just data plus things we do to that data. So we need good ways to hold it, organise it, and save it. And that is exactly our three topics. Collections are about holding lots of data in memory while the program runs. Class relationships are about how our objects connect to each other in a tidy way. And File Input-Output is about saving data onto the disk so it survives after the program closes.

So even though it looks like three separate topics, keep that thread in your mind - it is all data management. By the end of today, you will have the full toolkit for handling data in any Java program, and especially in the Spring projects you will build later.

DELIVERY TIP

Set a friendly, unhurried tone. This is a long session, so promise them it is built from small, simple ideas - nothing scary.

Slide 2 - Agenda

ON SCREEN

Five numbered items with icons: Collections framework, sorting and choosing, class relationships, abstract vs interface, basic File I/O.

SAY (read this aloud)

Here is our plan for the session, so you can see the whole journey. First, the Collections framework - the three big families called Lists, Sets, and Maps. Then how to sort them and how to choose the right one for a job. Then we change gears and talk about class relationships - the famous 'is-a' versus 'has-a' idea. Then we compare abstract classes and interfaces, which beginners often mix up. And finally, basic File Input-Output - reading from and writing to files.

Now, one promise. There are short pieces of code on these slides, but please do not panic and try to copy them down frantically. They are there just so you can see the idea. This is a reading-and-understanding session, not a typing session. Relax and follow along.

Slide 3 - Why collections exist

ON SCREEN

Three cards: store 500 names; remove the item at index 200; check if 'Ravi' is already present.

SAY (read this aloud)

Before we learn collections, let me convince you why we even need them. You already know arrays, right? An array is a fixed row of boxes. Arrays are fine, until they are not - and the three cards on screen show exactly where they fall apart.

First card: you make an array for 500 student names. Great. But then a 501st student joins. Now what? An array cannot grow - you decided its size at the start and you are stuck. You would have to make a whole new bigger array and copy everything over by hand. Annoying.

Second card: you want to remove the student at position 200. With an array, removing one from the middle means you have to manually shift every single student after it one step to the left, to fill the gap. Imagine doing that for thousands of records. With a collection? One line - list dot remove. Done.

Third card: you want to check, 'is Ravi already in my list?' With an array, you write a loop and check each one. With a collection, again, one method - contains - and it tells you.

So here is the bottom line. Arrays are fixed in size and have no built-in helpers. Collections, on the other hand, can grow and shrink freely, and they come loaded with handy methods. That is why they exist - to take all that painful manual work off your plate.

Slide 4 - The Collections hierarchy

ON SCREEN

A colour-coded tree: Iterable at the top, then Collection branching into List, Set, and Queue. Map is shown separately in its own box.

SAY (read this aloud)

This slide is the family tree of collections, and I want you to keep this mental map, because everything else fits into it. Let me read it from the top.

Right at the top is something called Iterable. The only thing you need to know is that 'iterable' means 'you can loop over it with a for-each loop'. Below that is Collection, which is the parent of most of what we use.

Collection splits into three families. First, List - a List keeps things in order, lets you grab items by their position number, and allows duplicates. Second, Set - a Set's special rule is no duplicates allowed, ever. And third, Queue - which handles items at the two ends, like a line of people.

Now look at the box sitting off to the side - Map. And here is the one thing I really want you to remember from this slide: Map is NOT part of the Collection family. Why? Because all the others store single items - one name, one number. But a Map stores things in pairs - a key and a value, like a word and its dictionary meaning. Because it works in pairs, it lives in its own separate family.

So, three families under Collection - List, Set, Queue - and Map standing on its own. Got that

picture? Good.

EMPHASISE

The 'Map is not a Collection' point is the most common interview trap from this topic. Say it slowly and maybe ask them to repeat it back.

Slide 5 - Generics

ON SCREEN

Two code panels: a raw list with no type that fails at runtime, versus a typed list that catches the error while typing. Three benefit cards.

SAY (read this aloud)

Before we go deeper, one quick but important idea - generics. That is the angle-bracket thing, like List of String. Let me show you why it matters with the two panels.

On the left is the old style, with no type specified. We make a list, we add the word 'Ravi', and then - oops - we accidentally add the number 101. The compiler says nothing. It happily allows it. The bug stays hidden, quietly waiting. Then later, when we pull an item out expecting a word, the program crashes. And it crashes at runtime - maybe in front of a customer, maybe in production. The worst possible time.

On the right, we add the type - List of String, meaning 'this list holds only words'. Now when we try to add the number 101, the compiler immediately stops us, right there, while we are still typing. The mistake is caught the instant we make it.

Think of it like a labelled jar in your kitchen. If a jar says 'Sugar' on it, you are not going to accidentally pour salt into it - the label protects you. Generics are that label on your collection.

So the three benefits are: errors caught early at compile time, no need to manually convert types, and your code documents itself because anyone can see what the list holds. The rule is simple - always put the type in the angle brackets. Always.

Part 1 - Lists

Slide 6 - ArrayList, the default List

ON SCREEN

Definition of a List, then two panels: everyday operations and a small performance table. A yellow gotcha box.

SAY (read this aloud)

Let's meet our first and most-used collection - the ArrayList. A List, remember, keeps order, lets you use position numbers, and allows duplicates. ArrayList is the workhorse you will reach for ninety percent of the time.

Here is how it works inside, and it is worth knowing. An ArrayList is secretly backed by a normal array. It starts with room for about ten items. When it fills up, it quietly makes a new array about

one and a half times bigger, copies everything over, and throws the old one away. You never see this happen - it just grows for you. Magic, but now you know the trick behind the magic.

The left panel shows the everyday methods - add, add at a position, get to read, set to replace, contains to check, and remove. The right panel shows the speed of each, and there is a clear story here. Getting an item by its position is instant - lightning fast. Adding at the end is fast. But adding or removing in the middle is slow, because remember, everything after has to shift over. And searching with contains is slow too, because it checks one by one.

So the simple guidance: use ArrayList when you mostly read and rarely insert in the middle. That is most situations, which is why it is your default.

EMPHASISE

Read the gotcha box out loud - it bites everyone once. For a list of Integers, calling remove with the number 1 removes the item at POSITION 1, not the value 1. To remove the value, you write remove of Integer dot valueOf 1. Tell them: 'remember this, you will hit it.'

Slide 7 - LinkedList, nodes and a Deque

ON SCREEN

A diagram of three linked nodes with prev and next pointers and HEAD/TAIL labels. Two panels: queue and stack operations, and a performance table.

SAY (read this aloud)

Now meet the ArrayList's cousin - the LinkedList. It does the same job, holding a list of things, but it is built in a completely different way, and that difference is the whole point.

Instead of one solid block like an array, a LinkedList is a chain. Picture a group of people holding hands. Each person - we call them a node - holds the data, and holds the hand of the person before and the person after. Look at the diagram - each box points to its neighbour on both sides.

Here is why that matters. If you want to add someone into the middle of a chain of people holding hands, you just have two people let go and grab the new person's hands instead. Nobody else has to move. So inserting is quick once you are at the spot. But - and here is the catch - to GET to the middle, you have to start at one end and walk hand by hand, counting, until you reach the position. So reading by position is slow.

Compare that to the ArrayList: ArrayList is instant to read by position but slow to insert in the middle. LinkedList is the opposite. They are mirror images.

One bonus - a LinkedList can also act as a queue, like a line at a shop, first in first out, or as a stack, like a pile of plates, last in first out. The panels show those methods - offer and poll for a queue, push and pop for a stack.

EMPHASISE

Mention the modern advice on the slide: for a pure queue or stack, ArrayDeque is actually better than LinkedList because of cache performance. Use LinkedList only when you truly need list-plus-queue behaviour together.

Slide 8 - ArrayList vs LinkedList

ON SCREEN

A decision table comparing the two across random access, insertion at beginning and end, memory, and cache performance, with a Winner column.

SAY (read this aloud)

So now the natural question - which one do I actually use? Let's settle it with this table, row by row.

Reading an item by its position number? ArrayList is instant, LinkedList is slow. ArrayList wins. Inserting or deleting at the very beginning? LinkedList just re-links the front, ArrayList has to shift everything. LinkedList wins that one. Adding at the end? Both are fast - it is a tie. Memory used per item? LinkedList needs extra space for all those hand-holding links, so ArrayList is leaner. And cache performance - this is a bit technical, but because ArrayList keeps everything together in one block, the computer's memory loves it; LinkedList's scattered nodes are slower. ArrayList wins.

So look at the winner column - ArrayList wins most rows. And that gives us our simple real-world rule: use ArrayList by default. Only reach for LinkedList in the specific case where you are constantly adding and removing at the ends. Honestly, about ninety percent of the time in real code, ArrayList is the right answer. So when in doubt, ArrayList.

Slide 9 - Sorting: Comparable vs Comparator

ON SCREEN

Two panels: Comparable implemented inside the Employee class with compareTo, and Comparator written outside as lambdas for different orderings.

SAY (read this aloud)

Let's talk about sorting. Here is the thing - Java knows how to sort numbers and how to sort words alphabetically, because those have an obvious order. But what about your own objects? If you have a list of Employees, Java has no idea whether you want them sorted by name, by salary, or by age. So you have to tell it. And there are two ways to do that.

The first way is called Comparable, shown on the left. Here, the Employee class itself decides its own natural order, by including a method called compareTo. In this example, Employees naturally sort by salary. The way compareTo works: it compares this object to another and returns a negative number, zero, or a positive number, meaning 'I come before', 'we are equal', or 'I come after'. Use this when there is one obvious, natural way to sort - and when you own the class so you can edit it.

The second way is called Comparator, on the right. This is sorting logic written outside the class, and it is far more flexible. With short lambda expressions you can sort by name, or by salary highest-first, or by department and then by salary. You can have as many different sort orders as you like.

So the simple rule: Comparable is the one built-in natural order. Comparator is for when you need many different orders, or when you cannot edit the class - like sorting Strings or Integers in a

custom way.

Slide 10 - The Collections utility class & list creation

ON SCREEN

Left panel: static helpers from `java.util.Collections`. Right panel: a small table comparing `Arrays.asList`, `List.of`, and `new ArrayList`. A yellow box on the Collections vs Collection confusion.

SAY (read this aloud)

Two handy practical things on this slide. First, on the left - there is a helper class called `Collections`, with an 's' on the end, and it is a treasure chest of free tools that work on any list. Want to sort a list? `Collections dot sort`. Reverse it? `Collections dot reverse`. Shuffle it randomly, like shuffling cards? `Collections dot shuffle`. Find the biggest or smallest? `max` and `min`. Many freshers never discover this class and reinvent these by hand - so now you know, just use it.

Second, on the right - there are three ways to create a list, and they behave differently, which surprises people. `Arrays dot asList` gives you a fixed-size list - you can change items but you cannot add or remove. `List dot of`, from Java nine, gives you a completely locked list - you cannot change anything at all. And `new ArrayList` gives you a fully flexible one you can do anything with. If you ever need to make a locked one flexible, just wrap it - `new ArrayList`, passing the locked one inside.

And one warning that the yellow box highlights. There is `Collections` with an 's' - the helper class - and `Collection` without an 's' - the interface, the family parent. They sound identical when spoken, but they are two different things. So when you see it written, check for that 's'.

Part 2 - Sets and Maps

Slide 11 - Set & how HashSet works

ON SCREEN

Definition of a Set, a panel showing membership in $O(1)$, and a panel explaining the hashing mechanism. A yellow box on the `hashCode/equals` contract.

SAY (read this aloud)

Now the second family - Sets. A Set has two simple rules: no duplicates allowed, and there are no position numbers. Its superpower is answering one question incredibly fast: 'have I seen this before?'

Let me show you why that is useful. Suppose you are tracking cities you have visited. With a List, every time you want to check 'have I been to Mumbai?', you scan the whole list - slow if the list is huge. With a Set, the answer comes back instantly, no matter how big it is. And if you try to add Mumbai twice, the Set just quietly ignores the duplicate. Perfect for this job.

How does HashSet do it so fast? Here is the idea in plain terms. When you add 'Ravi', the Set runs his name through a little maths function called a hash, which produces a number, and that number

decides which 'bucket' or shelf to put him on. Later, when you ask 'is Ravi here?', it runs the same maths, gets the same number, goes straight to that one shelf, and checks only there. It never searches the whole thing. That is the trick behind the speed.

There is one side-effect of this shelf system, and it confuses beginners: a Set has no predictable order. Ravi might land on shelf three and Anita on shelf one, so when you loop through, Anita comes out first even though you added Ravi first. So just expect - a Set does not remember the order you added things.

EMPHASISE

Read the yellow box carefully - it is subtle but important. If you put your OWN objects in a HashSet, you must override both hashCode and equals, and keep them consistent. If two objects are 'equal', they must produce the same hash number - otherwise they land on different shelves and the Set breaks. Rule: override one, override the other.

Slide 12 - Choosing a Set implementation

ON SCREEN

Three cards - HashSet, LinkedHashSet, TreeSet - a TreeSet code example, and a performance table.

SAY (read this aloud)

There are three kinds of Set, and choosing between them is all about one question - do you care about order? Let me give you the three options.

HashSet is the fastest, and it gives no order at all - things come out in that random shelf order we just discussed. It is the default choice when you just need uniqueness and speed.

LinkedHashSet does one extra thing - it remembers the order you added items in. So if you add banana, apple, cherry, that is the order you get back. It costs a tiny bit more memory for that, but it is worth it when order matters to you.

And TreeSet keeps everything automatically sorted, all the time. Add numbers in any jumbled order, and they come out neatly sorted. The little example on screen shows exactly that - we add some scores in random order with a duplicate, and TreeSet gives them back sorted, with the duplicate dropped. It is a little slower because it does the sorting work, but you get permanent order for free.

So, the easy summary - and write this in your notes: HashSet for pure speed, LinkedHashSet to remember insertion order, TreeSet to keep things sorted.

Slide 13 - Map & HashMap

ON SCREEN

Definition of a Map, a HashMap code example with put, get, getOrDefault, containsKey, and a note box with the frequency-counter pattern.

SAY (read this aloud)

Now the family I personally use the most in real projects - the Map. Pay close attention here, because Maps are everywhere.

A Map stores data in pairs - a key and a value. Each key is unique, and each key points to one value. The best way to understand it is with everyday examples. A dictionary is a map - the word is the key, the meaning is the value. A phone contacts list is a map - the name is the key, the number is the value. A class register is a map - the student ID is the key, the grade is the value. You look something up by its key and instantly get its value.

HashMap uses that same fast hashing trick from Sets, so look-ups are instant. In the example, we put 'Mumbai' with a population number. Notice when we put 'Mumbai' again with a new number, it updates the old one rather than adding a second Mumbai - because keys are unique. `get` fetches the value for a key, or gives null if the key is not there. `getOrDefault` is safer - it gives a fallback instead of null. And `containsKey` just checks if a key exists.

Look at the note box - this little pattern is gold, you will use it constantly: counting things. For each word in some text, you do put the word with `getOrDefault` of that word, zero, plus one. In one line, you have counted how many times each word appears. Keep that pattern in your back pocket.

Slide 14 - Iterating Maps

ON SCREEN

Two panels: `entrySet` (and `forEach`) on the left, `keySet/values` on the right with a 'don't do this' warning. A small table of HashMap, LinkedHashMap, TreeMap ordering.

SAY (read this aloud)

So how do you loop through a Map? This trips beginners up, so let me give you the clean ways.

The best way, on the left, is `entrySet`. Why is it best? Because it hands you the key and the value together, in one go, in a single pass. That is efficient. And even cleaner in modern Java is `forEach` with a little lambda that just takes the key and value directly - very readable.

On the right are two other ways - `keySet`, when you only need the keys, and `values`, when you only need the values. Those are fine for those specific cases.

But read the warning, because this is a classic beginner mistake: do not loop over the keys, and then inside the loop call `get` for each key to fetch its value. Think about why that is wasteful - for every single item, you are doing two look-ups instead of one. If you need both the key and the value, just use `entrySet`, which gives you both at once.

And the little table at the bottom mirrors what we learned with Sets - there is HashMap with no order, LinkedHashMap which remembers insertion order, and TreeMap which keeps the keys sorted. Same three flavours, same idea.

Part 3 - Class Relationships

Slide 15 - Inheritance (IS-A)

ON SCREEN

Two panels: inherit-and-override with Animal and Dog, and polymorphism plus the super keyword.

SAY (read this aloud)

Okay, we are changing topics now - from storing objects to how objects relate to each other. The first relationship is the famous 'is-a' relationship, also called inheritance.

It is the most natural idea in the world. A dog is an animal. A car is a vehicle. A manager is an employee. In code, we say Dog extends Animal, and that means Dog automatically gets everything Animal has - if Animal can eat and sleep, Dog can eat and sleep for free, without rewriting it. And Dog can add its own things, like bark, and can also change, or 'override', an inherited behaviour to do it its own way.

Why is this powerful? Because of something called polymorphism - a big word for a simple idea. Look at the right panel - we can write Animal a equals new Dog. We are treating the dog as an animal, which is fine, because a dog is an animal. And when we call eat, Java is smart enough to run the Dog's version. This lets us treat many different types - dogs, cats, birds - all together as animals, and each still behaves correctly.

You will also see the word super. super in brackets calls the parent's constructor, and super dot something calls the parent's version of a method - handy when you want to reuse what the parent did and just add a little extra.

EMPHASISE

Two practical rules to stress. One: always write the at-Override annotation when overriding - if you make a typo in the method name, the compiler will catch it instead of silently creating a useless new method. Two: only use inheritance for a real 'is-a' relationship - not just to grab some code you like. For grabbing code, use composition, which is the next slide.

Slide 16 - Composition vs Aggregation

ON SCREEN

Two columns: Composition with House creating a Room (strong ownership), and Aggregation with Department referencing Employees (weak ownership).

SAY (read this aloud)

The second relationship is 'has-a' - one object contains another. A house has rooms. A car has an engine. A department has employees. Simple. But there are two flavours of 'has-a', and the difference is about ownership - and this is the part students most often mix up, so let me make it really clear.

The first flavour is composition - strong ownership. Look at the House on the left. The House creates its own rooms, inside itself. The rooms belong completely to that house. And here is the test - if you demolish the house, the rooms are gone too. A room cannot exist on its own, floating in the air, without a house. They live and die together. That is composition.

The second flavour is aggregation - weak ownership. Look at the Department on the right. A

department has employees, but the employees existed before they joined, and if the department is shut down tomorrow, the employees do not vanish - they just move to another department. The department groups them, but does not own their existence.

So here is the one simple question that tells you which is which - and I want you to actually use this test: can the inner thing exist on its own, without the outer thing? If yes, like an employee without a department, it is aggregation. If no, like a room without a house, it is composition. Just ask that question.

Slide 17 - Favour composition over inheritance

ON SCREEN

A problem about a Logger needing to both write a file and connect to a database, then a composition solution, and four benefit cards.

SAY (read this aloud)

Here is a piece of advice you will hear throughout your whole career as a programmer: favour composition over inheritance. That sounds abstract, so let me make it concrete with the example on screen.

Suppose you want to build a Logger - something that records messages. And you want it to both write to a file AND save to a database. With inheritance, you would want to extend both a FileWriter and a DatabaseConnector. But here is the wall you hit - Java does not allow a class to extend two parents. You simply cannot inherit from both. Dead end.

Composition solves it instantly and elegantly. Instead of trying to BE a FileWriter and a DatabaseConnector, the Logger simply HAS one of each, as fields inside it. Then its log method just asks each one to do its part. No conflict, no limit - you can compose as many helpers as you want.

The four cards list why this is better: no single-parent limit; you can swap pieces at runtime, like switching the file-writer for a network-writer; your classes stay small and focused on one job; and it is much easier to test, because you can slot in fake versions of the helpers.

So the rule of thumb: inherit only when it is truly an 'is-a' relationship. When you just want to reuse some behaviour, compose - put the helper inside as a field. Most of the time, composition is the right answer.

Slide 18 - Abstract class vs Interface

ON SCREEN

Two code panels - an abstract Vehicle class and a Saveable interface with multiple implements - and a comparison table.

SAY (read this aloud)

Now, two tools that beginners constantly confuse - abstract classes and interfaces. Both are ways to define a kind of contract for other classes to follow. Let me clear up the difference.

An abstract class, on the left, is a half-finished class. Look at Vehicle - it has a fully-written start

method that all vehicles share, but it also has an abstract method, `mileage`, with no body, that each specific vehicle must fill in. So an abstract class gives you some ready-made parts plus some blanks to fill. And you cannot create one directly - you cannot say `new Vehicle`, because it is incomplete. You must extend it.

An interface, on the right, is more like a pure list of capabilities. `Saveable` says 'anything that is saveable must have a `save` method'. The huge advantage is that a class can implement many interfaces at once - here, `Report` is both `Printable` and `Saveable`. Remember, a class can only extend one parent, but it can implement many interfaces. So interfaces are how Java gives you multiple capabilities.

The table sums it up. Quick way to decide: use an abstract class when your classes are closely related and share both data and behaviour - like different kinds of vehicles. Use an interface when you are giving a capability to possibly unrelated classes - like '`comparable`' or '`printable`' or '`runnable`'. Those are abilities, not family ties.

Part 4 - Basic File I/O

Slide 19 - Why File I/O

ON SCREEN

The persistence bridge explained, two columns for byte streams and character streams, and a note on the `File` class.

SAY (read this aloud)

Last topic - File Input-Output, or File I-O. And this connects right back to where we started the day. Collections hold data in memory, which disappears when the program ends. Files solve that by saving data onto the disk, where it stays. No database server needed - just a simple file. Perfect for things like settings, logs, and exporting reports.

Now, Java gives you two families of tools for files, and it is important to pick the right one. The first is byte streams - these are for binary files, things that are not plain text, like images, music, and PDFs. The second is character streams - these are for text files, and they handle the letters and encoding for you.

Here is the simple rule of thumb: about ninety percent of the time you will be working with text - text files, CSV files, config files. So reach for character streams. They are the ones that understand letters.

And a quick but important clarification about a class called `File`. The `File` class represents a path - like a label or an address for a file. It can tell you things - does this file exist, how big is it - and it can create or delete files. But here is the catch that surprises people: the `File` class does NOT read or write the contents. It is just the address on the envelope, not the letter inside. To read and write the contents, we use the streams, which is the next slide.

Slide 20 - Buffered text I/O

ON SCREEN

Two panels: writing with `BufferedWriter/PrintWriter`, and reading line by line with `BufferedReader`. A key box on the wrapping idea.

SAY (read this aloud)

This is the practical, everyday way to read and write text files. There is one core idea here, and it is called wrapping. Let me explain wrapping with an analogy first.

Imagine you are moving books from one room to another. You could carry them one at a time - one trip per book - which is painfully slow. Or you could put a box around them, fill the box, and carry many at once. That box is a buffer. A 'buffered' reader or writer puts a box around the basic file tool so it moves data in big efficient chunks instead of one tiny character at a time.

So look at the writing panel. We take a basic `FileWriter`, wrap it in a `BufferedWriter` for efficiency, and then wrap that in a `PrintWriter`, which adds the friendly `println` and `printf` methods you already know from printing to the screen. Notice it is all inside `try-with-resources`, so the file gets closed automatically - we will come back to that.

Now the reading panel. We wrap a `FileReader` in a `BufferedReader`, and then we read the file one line at a time with `readLine`, inside a while loop. Here is the neat trick - `readLine` gives back null when it reaches the end of the file, and that null naturally stops the loop. Then we split each line by commas to pull out the pieces.

So remember the wrapping picture: the `BufferedReader` wraps the `FileReader`, which opens the file. Each layer adds something - one opens, the other adds the efficient buffering and line reading.

DELIVERY TIP

Mention `Scanner` as the easy alternative they already know - it can read files too, with `nextLine` and `nextInt`. It is friendlier but slower for big files. `BufferedReader` is the professional choice for large files.

Slide 21 - try-with-resources, NIO.2 & Properties

ON SCREEN

Two panels: safe auto-closing with `IOException` handling, and the concise NIO.2 Files API. A note box with the Properties file pattern.

SAY (read this aloud)

Three finishing touches for files. Let me take them one at a time.

First, on the left - always close your files, safely. A file is a resource the operating system hands you, and if you forget to close it, you can leak it or even lock the file so nobody else can use it. The clean solution, just like we will see in JDBC, is `try-with-resources` - you declare the file in the brackets after `try`, and Java guarantees to close it for you, even if an error happens. And always catch `IOException`, which is the error for file problems like 'file not found' or 'no permission'.

Second, on the right - the modern, shorter way, in a package called `java dot n-i-o dot file`. Look how short these are. `Files dot writeString` to write a whole file in one line. `Files dot readString` to

read the whole thing. Files dot readAllLines to get every line as a list. For new code, prefer this modern style; you will still see the older stream style in existing projects, so know both.

Third, the note box - the Properties class, for reading config files. You load a properties file, then ask for values by their key. Now here is something that should excite you - this is almost exactly how Spring Boot reads its application dot properties file. When Spring picks up your database URL or your server port, it is doing something very much like this. So you are seeing, right now, the machinery inside Spring's configuration. Cool, right?

Slide 22 - Collections + File I/O in one flow

ON SCREEN

A single larger code block: read a CSV into a List of Employee, sort and group with Collections, then write the result back to a new file.

SAY (read this aloud)

This slide is the grand finale - it shows everything we learned today working together in one real task. Let me narrate the story of this code, in three acts.

Act one - read. We open a CSV file with a BufferedReader, skip the first line because it is just the header row, then loop through every line. For each line, we split it by commas and build an Employee object from the pieces, and we add each one into an ArrayList. So now our file data is sitting neatly in a collection in memory.

Act two - process. Now that it is in a list, we can use all our collection tools. We sort the employees by salary, highest first, using a Comparator. And we build a HashMap that counts how many employees are in each department. Reading, sorting, grouping - all the things we learned.

Act three - write. Finally, we open a PrintWriter and write the processed results back out to a brand new file - a clean report.

Step back and look at what just happened. In one short program, we used the ArrayList, sorting, Comparator, HashMap, the BufferedReader, the BufferedWriter, and the PrintWriter. Every single thing from today, cooperating to read data in, transform it, and save it back out. That is real programming, and you now understand every piece of it.

Recap & Close

Slide 23 - Session recap

ON SCREEN

A six-cell grid: Lists, Sets, Maps, Class relationships, File I/O, and Golden rules.

SAY (read this aloud)

Let's take a breath and recap everything in one view. Look how much ground we covered. I will go cell by cell - just nod along inside your head.

Lists: ArrayList is your default, fast to read by position; LinkedList is fast at the ends and can act as

a queue. Comparable is the one natural order, Comparator is for flexible custom orders.

Sets: HashSet is fastest with no order, LinkedHashSet remembers insertion order, TreeSet keeps things sorted.

Maps: HashMap is fast and unordered, TreeMap is sorted; loop with entrySet to get key and value together; and for your own objects as keys, override hashCode and equals together.

Class relationships: 'is-a' is inheritance, 'has-a' is composition or aggregation; favour composition for reusing code; abstract classes are half-finished, interfaces are pure capability contracts.

File I-O: BufferedReader and BufferedWriter for text, PrintWriter for nice formatting, the modern Files class for short code, and Properties for config files.

And the golden rules across everything: always use generics with the angle brackets; ArrayList and HashMap are the right default most of the time; and always use try-with-resources when working with files.

Slide 24 - Closing

ON SCREEN

The closing blue slide: 'The complete data toolkit.'

SAY (read this aloud)

And that brings us to the end. Today you picked up the complete toolkit for managing data in Java - holding it in memory with collections, connecting it structurally with class relationships, and saving it to disk with file I-O.

Here is the thought I want to leave you with. In every Spring Boot project you build from here on, all three of these are quietly at work, every single day. The list that holds your database results - that is collections. The way your entity classes relate to each other - that is class relationships. The application dot properties file Spring reads when it starts up - that is file I-O. You did not just learn three topics today; you learned the foundation under everything that comes next.

Really well done, all of you. Let's open the floor for any questions.

DELIVERY TIP

Leave this slide up for Q&A. The two most common questions are 'when would I really use LinkedList?' and 'why must I override both hashCode and equals?' - both have dedicated slides you can flip back to.